Ombygging: spillhylla og spillsiden henter fra API-et

Steg for steg fra lagring i komponentene til GameApiClient, med id-er fra lagringen og DTO-er som gjøres om ved grensen

Etter ombyggingen henter spillhylla og spillsiden alle data via `GameApiClient` og HTTP. Ingen komponent bruker `GamesStorageAndQuery` lenger. Søk, filter og «Tilfeldig spill» virker som før, men på lista som er hentet fra API-et. Begge sidene viser laster, feil og tom liste.

Før og etter

```
FØR
Games.razor        → GamesStorageAndQuery            (rett i lagringen)
GamesDetails.razor → GamesStorageAndQuery

ETTER
Games.razor        → GameApiClient → HTTP GET /api/games      → endepunkt → ServerGamesService
    → GamesStorageAndQuery
GamesDetails.razor → GameApiClient → HTTP GET /api/games/{id} → endepunkt → ServerGamesService
    → GamesStorageAndQuery
                   ← BoardGame     ← GameDTO som JSON
```

Fire ideer bak ombyggingen

Les disse før du begynner. Hvert steg senere er en konsekvens av én av dem.

1. Komponentene kjenner bare `GameApiClient`. All kommunikasjon med API-et samles i én klasse. Sidene vet ikke at det finnes HTTP, adresser eller DTO-er. Det er kravet i oppgave 4, og det gjør at lagringen kan byttes uten at sidene endres.

2. DTO-en gjøres om ved grensen. API-et sender `GameDTO`, fordi det bare skal vise det klienten trenger. Resten av appen bruker `BoardGame`. Omgjøringen skjer ett sted, i `GameApiClient`, så ingen komponent trenger å vite at DTO-en finnes.

3. Lagringen deler ut id-er, ikke modellen. Før satte konstruktøren i `BoardGame` en ny id hver gang et objekt ble laget. Da kunne ikke klienten lage en kopi av et spill uten å bruke opp en id. Konstruktøren beholdes, men uten id-tildelingen. Det er lagringen som gir nye spill id, slik en database gjør.

4. Filter og tilfeldig valg virker på en ferdig liste. Spillene hentes én gang. Søk og «Tilfeldig spill» jobber på lista som allerede er hentet, uten nye kall. Logikken ligger i `GameFilter`, utenfor komponentene, så den kan testes direkte.

Rekkefølgen

Stegene går nedenfra og opp: modell, lagring, logikk, server, klient, sider og tester. Hvert lag bygger på laget under, så når du kommer til sidene, finnes alt de trenger allerede. Prosjektet kompilerer ikke helt før steg 8 er gjort, fordi sidene fortsatt bruker de gamle metodene underveis.

Steg	Fil	Hva
1	`Models/BoardGame.cs`	Fjern telleren og id-linjen i konstruktøren
2	`Data/GamesStorageAndQuery.cs`	Lagringen gir id. Tilfeldig valg flyttes ut
3	`Filtering/GameFilter.cs`	Tar imot hvilken som helst liste. Får tilfeldig valg
4	`ServerServices/ServerGamesService.cs`	Hent ett spill, og én felles omgjøring til DTO
5	`Program.cs`	Endepunktet `GET /api/games/{id}`
6	`Client/GameApiClient.cs`	Returnerer `BoardGame`, og henter ett spill
7	`Pages/AdminGames.razor`	Én linje: ny type på lista
8	`Pages/Games.razor` og `GamesDetails.razor`	Henter via API-et, med tre tilstander
9	`AK2Test/Oppgave1.cs`	Testene bruker `GameFilter`

Steg 1: BoardGame mister telleren

Fjern feltet `_nextId` og linjen `Id = _nextId++;` i konstruktøren. Fjern også `using Microsoft.AspNetCore.Server.Kestrel.Core;`, som ikke brukes. Konstruktøren beholdes ellers som den er.

Models/BoardGame.cs (hele filen)

```
using System.ComponentModel.DataAnnotations;

namespace AK2.Models
{
    public class BoardGame
    {
        public int Id { get; set; }
        public DateTime? StartedAt { get; set; }
        public string Name { get; set; } = "";
        public int MinPlayers { get; set; }
        public int MaxPlayers { get; set; }
        public TimeSpan PlayTime { get; set; }
        public string Category { get; set; } = "";

        [Range(1, 6, ErrorMessage = "Terningkastet må være fra 1 til 6")]
        public int Rating { get; set; } // dice roll, 1-6
        public string Description { get; set; } = "";

        public BoardGame(string name, int minPlayers, int maxPlayers, string category)
        {
            Name = name;
            MinPlayers = minPlayers;
            MaxPlayers = maxPlayers;
            Category = category;
        }
    }
}
```

Konstruktøren gjør nå bare én ting: setter verdiene den får inn. Å lage et `BoardGame` har ingen bieffekt lenger, så det kan gjøres så mange ganger det trengs, for eksempel når en kopi lages ut fra API-et eller når spillekvelden leses fra nettleseren.

`Id` er fortsatt en vanlig egenskap med `set`. Den settes etter at objektet er laget, med objektinitialisering: `new BoardGame(...) { Id = 3 }`. Klammene etter parentesen setter egenskaper som ikke er med i konstruktøren.

> Alle som kaller konstruktøren, gjør det med de samme fire verdiene som før, så ingen kall går i stykker. Men ingen av dem får lenger en id automatisk. I prosjektet ditt lager bare `GamesStorageAndQuery` nye spill, og den får ansvaret for id-ene i neste steg.

Steg 2: Lagringen deler ut id-er

Lagringen får sin egen teller. Den er ikke `static`, fordi lagringen er registrert som singleton: det finnes bare én lagring, og dermed én teller.

Data/GamesStorageAndQuery.cs, øverst i klassen

```
public List<BoardGame> boardGames { get; set; } = new();

// The storage hands out ids, like a database would
private int nextId = 1;
```

Data/GamesStorageAndQuery.cs, CreateGameAsync

```
// Create
public Task CreateGameAsync(string name, int minPlayers, int maxPlayers, string category)
{
    boardGames.Add(new BoardGame(name, minPlayers, maxPlayers, category)
    {
        Id = nextId++
    });

    return Task.CompletedTask;
}
```

Spillet lages med konstruktøren som før, og lagringen setter id-en i klammene etterpå. `nextId++` gir den nåværende verdien og øker telleren etterpå, så første spill får 1, neste 2, og så videre.

Metoden er ikke lenger `async`. Den har ingenting å vente på, så den returnerer en ferdig `Task` med `Task.CompletedTask`. Den som kaller, kan fortsatt bruke `await`.

Slett også `GetRandomGameAsync`. Tilfeldig valg flyttes til `GameFilter` i neste steg, fordi det skal velge fra lista sidene har hentet, ikke fra lagringen.

Steg 3: GameFilter tar imot hvilken som helst liste

To endringer: parameteren blir `IEnumerable<BoardGame>` i stedet for `List<BoardGame>`, og klassen får `GetRandomGame`.

Filtering/GameFilter.cs (hele filen)

```
using AK2.Models;

namespace AK2.Filtering
{
    public static class GameFilter
    {
        public static IEnumerable<BoardGame> FilterGames(string nameFilter,
    IEnumerable<BoardGame> boardGames)
        {
            if (string.IsNullOrWhiteSpace(nameFilter))
                return boardGames;

            var filter = nameFilter.Trim();

            if (int.TryParse(filter, out int players))
            {
                return boardGames.Where(g =>
                    g.MinPlayers <= players && players <= g.MaxPlayers);
            }

            // Using StringComparison.CurrentCultureIgnoreCase to allow both uppercase and
    lowercase letters
            return boardGames.Where(g =>
                g.Name.Contains(filter, StringComparison.CurrentCultureIgnoreCase));
        }

        public static BoardGame? GetRandomGame(IReadOnlyList<BoardGame> boardGames)
        {
            if (boardGames.Count == 0)
                return null;

            int index = Random.Shared.Next(boardGames.Count);
            return boardGames[index];
        }
    }
}
```

`IEnumerable<BoardGame>` betyr «noe som kan gås gjennom». Både `List`, tabeller og resultatet av LINQ passer. Med `List` som parameter kunne filteret bare ta imot akkurat en liste, og spillhylla sender en annen type. Velg alltid den mest generelle typen metoden faktisk trenger.

`GetRandomGame` trenger `IReadOnlyList`, fordi den må vite antallet (`Count`) og hente på plass (`[index]`). Det kan ikke en ren `IEnumerable`.

Steg 4: ServerGamesService henter ett spill

Omgjøringen til `GameDTO` stod før inne i `GetAllGamesAsync`. Nå trengs den to steder, så den flyttes til en egen metode, `ToDto`. Da finnes reglene for hva som sendes ut, bare ett sted.

```
ServerServices/ServerGamesService.cs

public async Task<IReadOnlyList<GameDTO>> GetAllGamesAsync()
{
    var games = await query.GetAllGamesAsync();

    return games.Select(ToDto).ToArray();
}

public async Task<GameDTO?> GetGameByIdAsync(int id)
{
    var game = await query.GetGameByIdAsync(id);

    return game is null ? null : ToDto(game);
}

private static GameDTO ToDto(BoardGame game) => new(
    game.Id,
    game.Name,
    game.Category,
    game.MinPlayers,
    game.MaxPlayers,
    game.Rating,
    game.Description);
```

`GetGameByIdAsync` returnerer `null` når spillet ikke finnes. Den vet ingenting om HTTP. Det er endepunktet som gjør `null` om til 404.

Steg 5: Endepunktet GET /api/games/{id}

Legg det etter `MapGet("")` i `Program.cs`. Oppgave 4 krever det, med 404 når id-en ikke finnes.

```
Program.cs

gamesApi.MapGet("{id:int}", async (int id, ServerGamesService gameService) =>
{
    var game = await gameService.GetGameByIdAsync(id);

    return game is null ? Results.NotFound() : Results.Ok(game);
});
```

`{id:int}` gjør to ting: verdien i adressen havner i parameteren `int id`, fordi navnene er like, og bare heltall matcher. `/api/games/abc` treffer derfor ikke endepunktet.

Steg 6: GameApiClient returnerer BoardGame

Her ligger grensen fra idé 2. `GetGamesAsync` får inn `GameDTO` fra API-et og gir `BoardGame` videre. `GetGameByIdAsync` er ny.

```
Client/GameApiClient.cs, øverst

using AK2.Models;
using System.Net;
```

Client/GameApiClient.cs

```
public async Task<IReadOnlyList<BoardGame>> GetGamesAsync()
{
    var dtos = await http.GetFromJsonAsync<GameDTO[]>("api/games") ?? [];

    return dtos.Select(ToBoardGame).ToArray();
}

public async Task<BoardGame?> GetGameByIdAsync(int id)
{
    var response = await http.GetAsync($"api/games/{id}");

    if (response.StatusCode == HttpStatusCode.NotFound)
    {
        return null;
    }

    response.EnsureSuccessStatusCode();

    var dto = await response.Content.ReadFromJsonAsync<GameDTO>();
    return dto is null ? null : ToBoardGame(dto);
}

// The DTO is converted here, so components only work with BoardGame
private static BoardGame ToBoardGame(GameDTO dto) =>
    new(dto.Name, dto.MinPlayers, dto.MaxPlayers, dto.Category)
    {
        Id = dto.Id,
        Rating = dto.Rating,
        Description = dto.Description
    };
```

Hvorfor ikke `GetFromJsonAsync` i `GetGameByIdAsync`? Den kaster unntak ved alle feilkoder, også 404. Her skal 404 bety «finnes ikke» og gi `null`, mens andre feil, som 500 eller at API-et er nede, skal kaste unntak. Derfor sjekkes statuskoden først, og svaret leses etterpå med `ReadFromJsonAsync`. Det er samme mønster som i `DeleteGameAsync`.

`ToBoardGame` lager en kopi med konstruktøren for de fire verdiene den tar imot, og setter `Id`, `Rating` og `Description` i klammene. Id-en kommer fra API-et. Siden konstruktøren ikke lenger har en teller (steg 1), brukes ingen id opp.

Steg 7: AdminGames.razor får ny type

`GetGamesAsync` gir nå `BoardGame`, så lista på adminsiden må ha samme type. Tabellen bruker de samme egenskapene, så ingenting annet endres.

Components/Pages/AdminGames.razor, @code

```
private IReadOnlyList<BoardGame>? games;
```

Steg 8a: Spillhylla henter via API-et

1. Slett linjen `@inject Data.GamesStorageAndQuery gamesQuery;` øverst.
2. Rett søkefeltet: `/>` står inne i placeholder-teksten, så taggen lukkes aldri. Det må være `placeholder="Navn / antall spillere" />`.
3. Bytt meldingen «Ingen treff» med de fire tilstandene under.
4. Legg til feltene for laster og feil, og bytt `OnInitialized` med `OnInitializedAsync`.
5. La `ShowRandomGame` bruke `GameFilter`.

Games.razor, i stedet for «Ingen treff»

```
@if (isLoading)
{
    <p>Laster spill...</p>
}
else if (error is not null)
{
    <p style="color:red">@error</p>
}
else if (!games.Any())
{
    <p>Det finnes ingen spill i samlingen.</p>
}
else if (!filteredGames.Any())
{
    <p style="color:red">Ingen treff</p>
}
```

Rekkefølgen i `if`-kjeden er viktig: først om den laster, så om det ble feil, så om samlingen er tom, og til slutt om søket ga treff. En tom samling og et søk uten treff er to ulike ting for brukeren, og får derfor hver sin melding.

Games.razor, @code: feltene øverst

```
private IReadOnlyList<BoardGame> games = [];
private bool isLoading = true;
private string? error;
```

Games.razor, @code: i stedet for OnInitialized

```
protected override async Task OnInitializedAsync()
{
    GameNight.OnChange += OnGameNightChanged;

    try
    {
        games = await ApiClient.GetGamesAsync();
        ApplyFilter();
    }
    catch (HttpRequestException)
    {
        error = "Kunne ikke hente spillene. Prøv igjen senere.";
    }
    finally
    {
        isLoading = false;
    }
}
```

Games.razor, @code: i stedet for den gamle ShowRandomGame

```
private void ShowRandomGame()
{
    randomGame = GameFilter.GetRandomGame(games);
}
```

Tre ting å legge merke til:

- `ApplyFilter()` **kalles etter at spillene er hentet.** Kalles den før, filtreres en tom liste.
- `finally` **kjører uansett.** Både når hentingen lykkes og når den feiler, slutter siden å vise «Laster».

- **Bare `HttpRequestException` fanges.** Det er feilen `HttpClient` kaster når API-et ikke svarer eller gir en feilkode. Andre feil, som en programmeringsfeil, skal ikke skjules som «Kunne ikke hente spillene».

`ApplyFilter` trenger ingen endring. Den kaller allerede `GameFilter.FilterGames(nameFilter, games)`, og etter steg 3 godtar filteret lista.

Steg 8b: Spillsiden henter via API-et

1. Bytt `@inject Data.GamesStorageAndQuery gamesQuery` med `@inject GameApiClient ApiClient`.
2. Legg «laster» og «feil» foran `@if (game is not null)`, så den blir `else if`.
3. Legg til feltene, og bytt innholdet i `OnParametersSetAsync`.

GamesDetails.razor, øverst i markupen

```
@if (isLoading)
{
    <p>Laster spill...</p>
}

else if (error is not null)
{
    <p style="color:red">@error</p>
}

else if (game is not null)
{
```

GamesDetails.razor, @code: feltene

```
private BoardGame? game;
private bool isLoading = true;
private string? error;
```

GamesDetails.razor, @code: OnParametersSetAsync

```
protected override async Task OnParametersSetAsync()
{
    isLoading = true;
    error = null;
    game = null;

    try
    {
        game = await ApiClient.GetGameByIdAsync(Id);

        if (game is null)
        {
            Nav.NotFound();
        }
    }
    catch (HttpRequestException)
    {
        error = "Kunne ikke hente spillet. Prøv igjen senere.";
    }
    finally
    {
        isLoading = false;
    }
}
```

Spillsiden skiller mellom to situasjoner som begge gir «ingen spill»:

Situasjon	API-et svarer	GameApiClient gir	Siden viser
Id-en finnes ikke	404	`null`	«Fant ikke»-siden via `Nav.NotFound()`
API-et svarer ikke, eller feiler	Ingen svar, eller 500	`HttpRequestException`	Feilmeldingen

`isLoading`, `error` og `game` nullstilles øverst i metoden. Den kjører på nytt når `Id` i adressen endres, og da skal ikke det forrige spillet eller en gammel feilmelding henge igjen mens det nye hentes.

Steg 9: Testene bruker GameFilter

`FilterGames` finnes ikke lenger i `GamesStorageAndQuery`, så testene kompilerer ikke før de er endret. Mønsteret er det samme i alle seks: lag dummy-spillene, hent lista, og filtrer den med `GameFilter`.

AK2Test/Oppgave1.cs, øverst

```
using AK2.Filtering;
```

AK2Test/Oppgave1.cs: hjelpemetoden og den første testen

```
// Creates the dummy games and returns them as a list
private async Task<List<BoardGame>> GetDummyGamesAsync()
{
    await _dummy.CreateDummyGames();
    return await _query.GetAllGamesAsync();
}

[Fact]
public async Task FilterGames_WhenNoMatchOnName_ShouldReturnEmpty()
{
    // Arrange
    var games = await GetDummyGamesAsync();

    // Act
    var filter = GameFilter.FilterGames("q", games);

    // Assert
    Assert.Empty(filter);
}
```

Gjør de fem andre testene om på samme måte: `public async Task`, `var games = await GetDummyGamesAsync();`, og `GameFilter.FilterGames(verdi, games)` i stedet for `_query.FilterGames(verdi)`. Linjene `var games = _query.GetAllGamesAsync();`, som ikke ble brukt, forsvinner.

Hjelpemetoden `GetDummyGamesAsync` venter på `CreateDummyGames` med `await`. Uten `await` er det ikke sikkert at spillene er lagt inn før testen filtrerer. Testmetodene blir derfor `async Task`, og xUnit venter på dem.

Bygg og test

1. **Stopp appen helt og start den på nytt.** Hot Reload klarer ikke å bytte ut endringer i konstruktører, `Program.cs` og metodesignaturer, og da kjører en blanding av gammel og ny kode.
2. Bygg løsningen (**Build → Rebuild Solution**), og kjør testene i Test Explorer.
3. Gå gjennom sjekklisten under.

Sjekk	Forventet
Åpne `https://localhost:7259/api/games`	JSON med de åtte spillene, id 1–8
Åpne `https://localhost:7259/api/games/1`	JSON med Monopol
Åpne `https://localhost:7259/api/games/999`	404
Spillhylla	«Laster spill...» et øyeblikk, så alle spillene
Søk på tekst og på et tall	Riktige treff. «Ingen treff» når ingen passer
«Tilfeldig spill»	Navnet på et av spillene
Opprett et spill på adminsiden, gå til spillhylla	Det nye spillet er med, med neste ledige id
Spillsiden for et spill, og for `/games/999`	Spillet vises / «Fant ikke»-siden
Feiltilstand: sett feil port i `ApiBaseUrl` i `appsettings.json` og start på nytt	Spillhylla og spillsiden viser feilmeldingen og krasjer ikke. Husk å sette porten tilbake
Legg til et spill i spillekvelden, last siden på nytt	Spillekvelden står der fortsatt

API-et ligger i samme app som sidene, så det kan ikke stoppes alene. Feil port i `ApiBaseUrl` gir samme resultat for `GameApiClient`: ingen svarer på adressen.

> **Utenfor denne ombyggingen:** `POST /api/games` svarer i dag `204 No Content`. Oppgave 4 krever `201` med det nye spillet i svaret, så det endepunktet må endres senere.

Hva som er kontrollert

- Hele prosjektet ditt kompilerer etter ombyggingen, også testprosjektet.
- Alle 12 testtilfellene i `Oppgave1.cs` passerer med `GameFilter`.
- Dummy-spillene får id 1–8 som før, og et spill som opprettes via API-et, får id 9.
- API-et er startet og kalt: `GET /api/games` gir alle spill, `GET /api/games/1` gir 200, og `GET /api/games/999` gir 404.
- Et `BoardGame` kan fortsatt lagres og leses som JSON, slik spillekvelden i nettleseren trenger. `System.Text.Json` bruker konstruktøren fordi parameternavnene stemmer med egenskapene, og setter `Id` og resten etterpå.

Ikke kontrollert: sidene i nettleseren. Sjekklisten over er første gang de kjøres.